

KUBERNETES GPU SHARING PLATFORM

GShare

하나의 **GPU** 클러스터를 조직 전체가 나눠 씁니다

여러 조직 · 부서 · 사용자가 하나의 GPU 클러스터를 분할 (fractional) · 전용 (exclusive) 으로 나눠 쓰고 ,
크레딧으로 과금 · 관리하는 GPU 공유 플랫폼 .

분할 · 전용 혼용

크레딧 기반 과금

유휴 **GPU** 자동 회수

멀티 클러스터 연동

GPU 를 직접 쪼개는 대신 , 그 위에서 누가 · 얼마나 · 어떤 비용으로 쓰는지를 책임지는 제어 영역 .

왜 필요한가

GPU 는 비싼데 , 늘 누군가는 놀고 있습니다

기존 방식의 한계

- × 1 인 1 카드 전용 할당 → 점유 시간 대부분 GPU 유휴
- × 단순 time-slicing · 분할은 비용 귀속 · 한도 관리가 없음
- × 유휴 세션이 카드를 붙잡아 대기열만 길어짐
- × 부서 · 조직별 예산 통제 · 감사 추적이 어려움
- × 큰 작업이 들어갈 빈 카드가 조각나 사라짐 (단편화)

GShare 의 접근

- ✓ 분할 · 전용을 한 클러스터에서 혼용
- ✓ 점유율 (occupancy) 기반 과금 — 쓴 만큼 크레딧 차감
- ✓ 유휴 GPU 는 자동 일시정지로 즉시 회수 , 작업은 보존
- ✓ 계층 크레딧 · 예산 게이트 · 자원 정책으로 한도 관리
- ✓ 점유율을 고려한 bin-packing 으로 빈 카드 보존

핵심 가치

한눈에 보는 GShare

2 가지 모드

분할 vGPU · 전용 풀카드

4 계층 크레딧

시스템 → 조직 → 부서 → 개인

유휴 자동 회수

idle GPU 자동 일시정지

멀티 클러스터

외부 GPU 클러스터 연동

실제 GPU 분할은 HAMi(분할 vGPU) 와 전용 풀카드가 맞고 ,
GShare 는 그 위에서 배치 · 과금 · 회수 · 권한을 하나의 콘솔로 통합 운영합니다 .

분할 위에 얹은 운영 지능

점유율을 고려한 bin-packing

VRAM·코어 두 축의 잔여를 함께 보는 best-fit 배치. 빈 카드를 보존해 전용·대형 작업 수용력 유지.

유휴 자동 일시정지

유휴 (util $\approx$ 0) 세션을 종료가 아닌 자동 일시정지로 회수. 세션·볼륨·크레딧은 보존, 언제든지 재개.

무손실 일시정지

cuda-checkpoint 로 GPU 메모리 상태를 보존한 채 카드를 반환하고 재개 시 그대로 복원. 모델 재로딩 비용 0.

계층 크레딧 · 예산

시스템→조직→부서→개인 배분. 세션은 본인 지갑에서만 과금, 범위별 예산 게이트·월 리필.

자원 정책 · 한도

동시 실행 수·자원 총량·유휴 타임아웃을 정책으로 통제. 사용자→부서→조직→공통 순으로 해석.

대화형 세션

VSCode·JupyterLab·웹터미널을 1회용 링크로 즉시 접속. 볼륨·스냅샷·감사 로그까지 콘솔 한 곳에서.

세션 생성 한 번에 일어나는 일

요청은 허용 검사를 순서대로 통과해야 기동되고, 그때부터 점유율 기반 과금이 시작됩니다.

STEP 1

선택

오퍼링 (GPU 모델 ·
티어) · 이미지 · 볼륨을
고르면 예상 크레딧을
표시.



STEP 2

허용 검사

정책 한도 → 예산 →
크레딧 hold →
점유율을 고려한
VRAM 예약 순으로
통과.



STEP 3

기동

제어 영역이 CR 적용
→ operator 가 파드 ·
서비스 · 인그레스 생성.



STEP 4

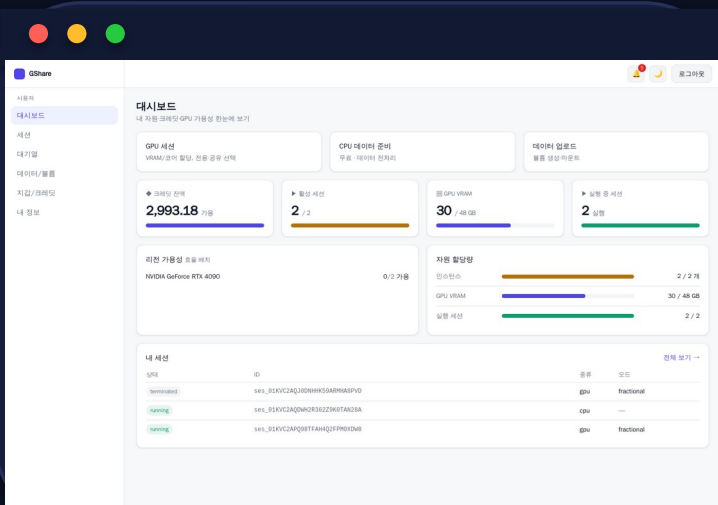
과금 · 회수

running 콜백부터
과금 시작, 유희 시
자동 일시정지로 GPU
반환.

데모 · 사용자 콘솔

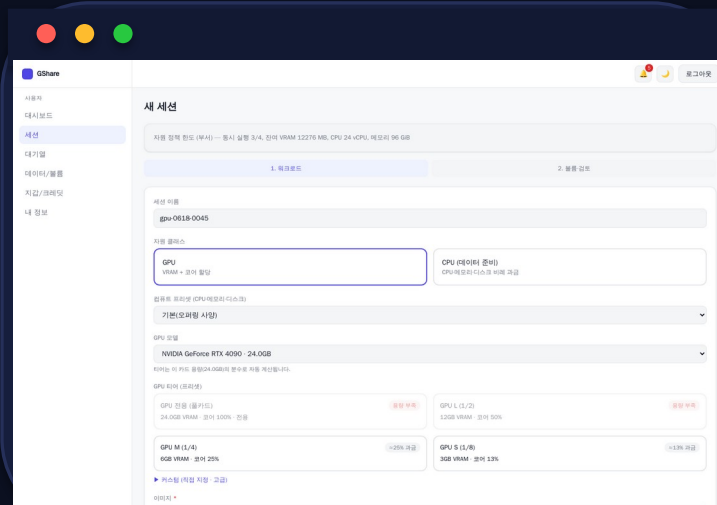
실제 콘솔 — 사용자 화면

세션을 직접 만들고, 접속하고, 크레딧을 확인하는 셀프서비스 콘솔.



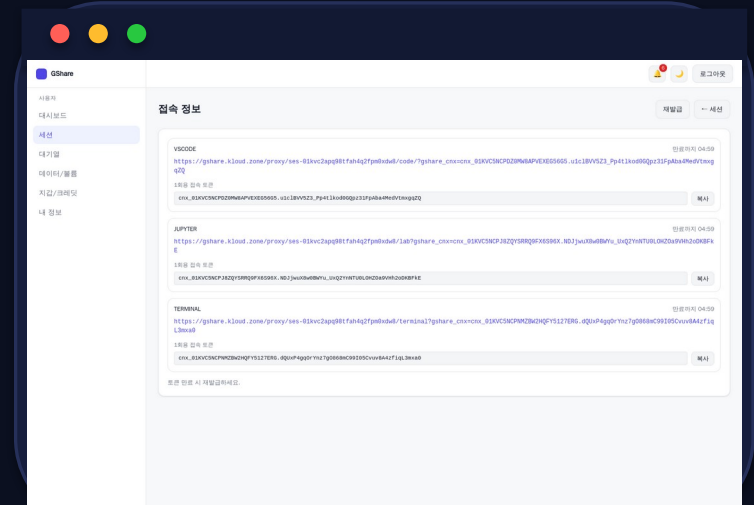
대시보드

내 자원 · 크레딧 · GPU 가용성 한눈에



새 세션 만들기

오퍼링 · 티어 · 이미지 선택 + 예상 크레딧



접속

VSCode·Jupyter· 터미널 1 회용 링크

제어 영역 · 실행 영역 분리

돈 · 상태 결정은 제어 영역이 책임지고 , 실제 쿠버네티스 실행은 클러스터별 operator 가 맡습니다 .

콘솔 (frontend · React SPA)

사용자 · 관리자 화면 (REST/SSE)

제어 영역 (backend · FastAPI)

gshare-api: 인증 · 권한 · 허용검사 · CRUD · gshare-worker: 과금 · 예산 롤업 · 대기열 · 리필

operator (Go · 클러스터별)

GShareSession CR → Pod·Service·Ingress 생성 (gshare-sessions ns)

HAMi + GPU 노드

실제 GPU 분할 (gpumem/gpucore) · 전용 풀카드

상태 저장소

postgres

상태 · 원장 (ledger)

redis

큐 · 토큰 · 락

멀티 클러스터

제어 영역이 kubeconfig
로 외부 GPU

클러스터에도 세션을 적용

전사부터 개인까지 , 범위에 맞게

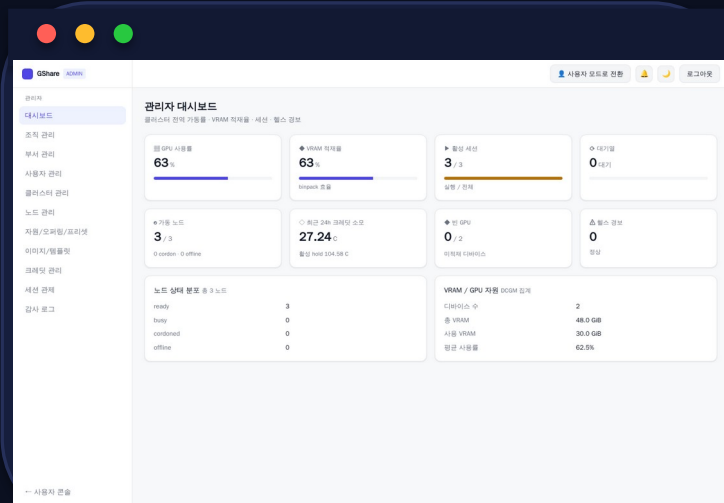
하나의 콘솔에서 역할 범위에 따라 메뉴와 데이터가 제한됩니다 .

역할	범위	할 수 있는 일
슈퍼관리자	전사	조직 · 오퍼링 · 정책 · 클러스터 · 노드 · 이미지 관리 , 크레딧 충전
조직관리자	자기 조직	부서 · 사용자 관리 , 조직→부서 예산 배분 , 조직 세션 관제 · 감사
부서관리자	자기 부서	부서원 관리 , 부서→개인 배분 , 부서 세션 관제
일반 사용자	본인	세션 생성 · 접속 , 지갑 · 크레딧 요청 , 볼륨 · 스냅샷

데모 · 관리자 콘솔

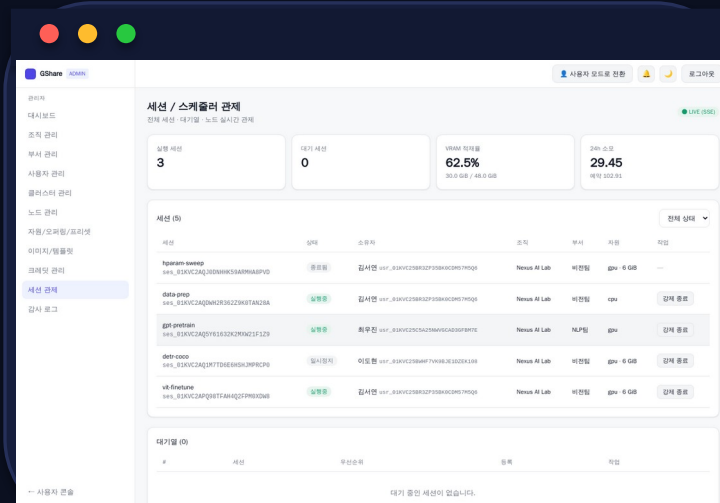
실제 콘솔 — 관리자 화면

클러스터 가동률 · 세션 · 크레딧을 하나의 콘솔에서 관제 · 배분 .



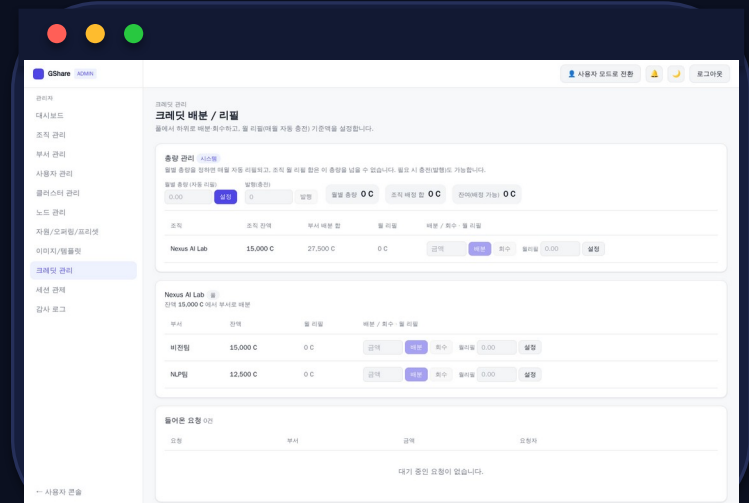
관리자 대시보드

가동률 · VRAM 적재율 · 헬스 정보



세션 관제

전체 세션 · 대기열 실시간 (LIVE SSE)



크레딧 배분

시스템 → 조직 → 부서 배분 · 월 리필

계층 크레딧으로 비용을 통제

크레딧은 위에서 아래로 배분되고 , 과금은 점유한 만큼 본인 지갑에서 차감됩니다 .



점유율 기반 과금

running 콜백부터 세션이 점유한 자원 · 시간만큼 크레딧 차감 — 쓴 만큼만 비용 .

예산 게이트

세션 생성 전 범위별 예산을 검사해 초과를 차단 · 경고 , 월 자동 리필 지원 .

크레딧 hold

기동 직전 예상 비용을 선점유 (hold) 해 과다 사용 · 중복 기동을 방지 .

도입

5 분 데모부터 멀티 클러스터까지

5 분 데모

Docker Compose

제어 영역과 콘솔을 띄워 화면 · 흐름을 바로 체험 · GPU 없이도 시연 가능 .

올인원 배포

GPU 클러스터

전체 스택 (콘솔 · 제어 영역 · operator · HAMi) 을 GPU 클러스터에 배포해 실제 세션을 운영 .

멀티 클러스터

kubeconfig 연동

하나의 제어 영역에서 외부 GPU 클러스터들에 세션을 적용 · 과금 · 관제 .

시작하기 가이드 · 사용자 / 관리자 매뉴얼 — github.com/boanlab/gShare

하나의 GPU 클러스터를 , 조직 전체가 , 통제 가능하게 .

GShare — 분할 · 전용을 한 클러스터에서 , 크레딧으로 과금하고 , 유희는 자동 회수 .

github.com/boanlab/gShare · Apache-2.0